Module 5 — MCP Servers

Lesson 5.2

Connecting existing MCP servers

Most teams should start by wiring up existing MCP servers (filesystem, GitHub, Slack, Postgres) before building their own.

Why it matters

The Anthropic and community ecosystems already have MCP servers for the systems you probably use most. Reaching for an existing server before writing one is a year of saved work.

How to add a server

Two equivalent paths:

Path 1 — `claude mcp` CLI

```
claude mcp add filesystem -- npx @modelcontextprotocol/server-filesystem ~/projects
claude mcp list
```

This writes to the right config file for you.

Path 2 — Edit `settings.json` directly

In `.claude/settings.json` (project) or `~/.claude/settings.json` (user-global):

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/me/projects"
      ]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

Key conventions:

- `command` + `args` form a subprocess that speaks MCP over stdio
- `env` injects env vars into the subprocess (use shell-style `${VAR}` to forward from your environment, don't hard-code secrets)
- After editing, restart `claude` to pick up new servers

For HTTP transport, the shape is different — the config provides a URL and optional auth headers. Check the official MCP docs for the exact key names against your installed version.

Common ready-to-use servers

Server	What it does
`@modelcontextprotocol/server-filesystem`	Expose a directory as resources Claude can read/write
`@modelcontextprotocol/server-github`	Issue/PR/repo operations via GitHub API
`@modelcontextprotocol/server-postgres`	Run read-only queries against a Postgres DB
`@modelcontextprotocol/server-slack`	Read messages, post to channels
`@modelcontextprotocol/server-puppeteer`	Browser automation
Linear, Notion, Sentry, others	First-party and community servers exist for many SaaS tools

Browse the canonical list at https://github.com/modelcontextprotocol/servers.

Scope choices

Where to add	When
Project `settings.json`	The server is project-relevant (e.g., a DB connection for this app)
User-global `~/.claude/settings.json`	The server is generally useful (e.g., GitHub, filesystem)
`settings.local.json`	The server config has personal secrets you don't want committed

Inspecting and managing servers

Once configured:

```
claude mcp list                    # show configured servers
claude mcp test <name>             # smoke-test a server
```

Inside a session, ask Claude to list what's available:

> *"List the MCP tools available right now and what each does."*

This is a great quick check that your setup is working.

Permissions for MCP tools

MCP tools show up in the permissions system like built-in tools, with names like `mcp__<server-name>__<tool-name>`. You can allow, ask, or deny them:

```
{
  "permissions": {
    "allow": ["mcp__filesystem__*"],
    "ask":   ["mcp__github__create_issue", "mcp__slack__post"],
    "deny":  ["mcp__postgres__execute_query"]
  }
}
```

Tighten permissions on tools with side effects (writes, posts, deletes). Reads can often be allow-listed.

Resource references in conversation

For servers that expose resources, you reference them by URI:

> *"Read the resource* `filesystem:///Users/me/projects/notes/today.md` *and summarize it."*

Or you can use the `@mention` syntax (if available in your version) — see `/help` for the exact form.

Troubleshooting

Symptom	Likely cause
Server doesn't appear in `/help` or `mcp list`	Not loaded — restart `claude` after editing config
Server appears but tools error	Auth/env vars not getting through — check `env` block and shell export
Server hangs on startup	Subprocess command is wrong; test it manually first
Tool calls fail with "not allowed"	Check `/permissions`; add to `allow`

Test the subprocess manually before plumbing into Claude:

```
npx -y @modelcontextprotocol/server-filesystem ~/projects
# Should print server handshake / wait for MCP protocol input on stdin
```

Try it

1. Add the filesystem MCP server scoped to your projects directory.
2. Restart `claude`. Confirm it appears in tools list.
3. Ask: "Use the filesystem MCP server to list the top 5 most recently modified files under ~/projects."
4. Add the github server if you use GitHub. Ask Claude to list issues in a small repo of yours.

Gotchas

- **Restart is required.** Almost every "the server isn't working" issue is a forgotten restart.
- **Don't pass secrets in `args`.** They show up in process listings. Use `env` with `${VAR}` substitution.
- **Servers that expose `Bash`-equivalent power need careful permission scoping.** Default-allow on a SQL server tool is effectively giving Claude write access to your DB.
- **Servers can be slow.** A first call may take seconds while the subprocess warms up.

Further reading

- 5.5 Auth and security
- examples/04-mcp-server-python — build your own

- Server gallery: https://github.com/modelcontextprotocol/servers

Next

→ 5.3 Build a Python MCP server